

Sprint Documentation

Sprint 13	Start Date: 16/03/2026	Final Date: 19/03/2026
Projetc Name:	civitas-frontend	
Ano: 5º	Análise e Desenvolvimento de Sistemas - AMS	
Team Members		
Karen valiao de abreu		

Sprint Backlog

Task#	Description	Start Date	Final date
108	Padronização de loading (skeleton) em páginas CRUD	16/03/2026	19/03/2026

Task Description

Task #	Description	Assigned To	Status	Estimated Hours	Logged Hours
108	Padronizar os estados de carregamento no frontend com skeleton visual consistente, substituindo carregamentos textuais simples.	Karen Valiao de Abreu	Finished	10 Horas	10 Horas

Sprint Description

★ Tarefa: Sprint 13 - Front - Padronização de loading (skeleton) em páginas CRUD

🎯 Objetivo

Padronizar os estados de carregamento no frontend com skeleton visual consistente, substituindo carregamentos textuais simples.

✓ Critérios de Aceite

- Componente de skeleton reutilizável criado para páginas CRUD.
- Páginas com Carregando... migradas para skeleton padronizado.
- Removida dependência de loading artificial por timer no layout.

- Loading representa estado real de requisição da API.
- Estado de erro e estado vazio continuam preservados.



Observações

- Não aplicar skeleton fixo fora do estado de loading.
- Reaproveitar o mesmo padrão em novos módulos.

Sprint Results

Durante a Sprint 13, foi realizada a padronização dos estados de carregamento nas páginas CRUD do sistema, com foco na substituição de abordagens textuais por uma representação visual estruturada baseada em skeleton loading.

A implementação teve como principal objetivo aprimorar a experiência do usuário, proporcionando maior clareza durante o carregamento de dados e reduzindo a sensação de espera. Para isso, foi desenvolvido um componente centralizado de loading (skeleton.tsx), projetado de forma reutilizável e adaptável a diferentes contextos da aplicação.

O componente foi estruturado em três variações principais: um skeleton genérico parametrizável, um skeleton específico para tabelas (utilizado nas páginas CRUD) e um skeleton voltado para formulários. Essa abordagem possibilitou a padronização visual do carregamento, mantendo coerência entre diferentes telas do sistema.

A implementação foi realizada utilizando React e Tailwind CSS, com destaque para o uso da classe `animate-pulse`, responsável por gerar uma animação leve e contínua. Tal escolha garantiu um equilíbrio entre qualidade visual e desempenho, evitando impactos negativos na renderização da interface.

Durante o processo, todas as páginas que utilizavam mensagens como “Carregando...” foram refatoradas para adotar o novo padrão visual. Além disso, foi removida a dependência de carregamentos artificiais baseados em temporizadores, assegurando que o estado de loading esteja diretamente associado às requisições reais da API.

Os estados de erro e de ausência de dados foram mantidos, preservando a lógica funcional previamente estabelecida e garantindo que a alteração fosse restrita ao aspecto visual do carregamento.

Como resultado, o sistema passou a apresentar maior consistência visual, melhor percepção de desempenho e uma estrutura de código mais organizada e reutilizável. A solução implementada também estabelece um padrão que pode ser facilmente reaplicado em novos módulos, contribuindo para a escalabilidade e manutenção do projeto.

Assinaturas

Documento assinado digitalmente



KAREN VALIAO DE ABREU

Data: 19/03/2026 10:06:10-0300

Verifique em <https://validar.iti.gov.br>